

DATA 260 - Homework 2

Student Name: Priyank Mehta

SJSU ID: 019152562

GitHub Repository: <https://github.com/priyankmehta5/data260-2562>

1. Personal Configuration

This report documents the implementation and evaluation of the DATA 260 Homework 2 system. The project extends the municipal transit incident application created for Homework 1 with a responsive user interface, a FastAPI CRUD service, a LangGraph supervisor workflow, and schema-validation experiments.

My SID4 is 2562, producing a DOMAIN_ID of 2. The assigned application port is 8762, calculated as $8000 + (2562 \text{ modulo } 900)$. Therefore, the assigned domain remains Municipal Transit Incidents.

Configuration value	Result
SID4	2562
PORT_BASE	8762
PREFIX	s2562
SEED	2562
VERIFY_SEED	262562
DOMAIN_ID	2
Assigned domain	Municipal transit incidents
Processor	Intel(R) Core(TM) i7-8750H CPU @ 2.20 GHz
Memory	15.86 GB RAM
Operating system	Windows 10
Dedicated GPU	NVIDIA GeForce GTX 1050 Ti
Integrated GPU	Intel(R) UHD Graphics 630
Local model	qwen3:1.7b
Implementation Commit	da28c955e4a11c21d367707d6735c829890e715e

2. Part 1 - Responsive Transit Incident Interface

2.1 Interface Extension

I extended the municipal transit incident application from Homework 1 into a responsive incident-management interface. The application now displays existing incidents, supports incident creation and editing, provides delete controls, and includes a search field.

The interface was implemented using the existing files in:

code/web_application/index.html

code/web_application/styles.css

code/web_application/script.js

```
<main class="app-shell"> <section class="form-panel"> <h2>Report a Transit Incident</h2>
<form id="incident-form"> <label for="title">Incident title</label> <input id="title"
name="title" required> <label for="content">Incident description</label> <textarea id="content"
name="content" required></textarea> <button type="submit">Add Incident</button> </form>
</section> <section class="incident-panel"> <h2>Transit Incidents</h2> <div id="incident-
list"></div> </section> </main>
```

The visible page retains the municipal transit incident domain and displays my name and SJSU ID. The layout changes automatically for smaller screens.

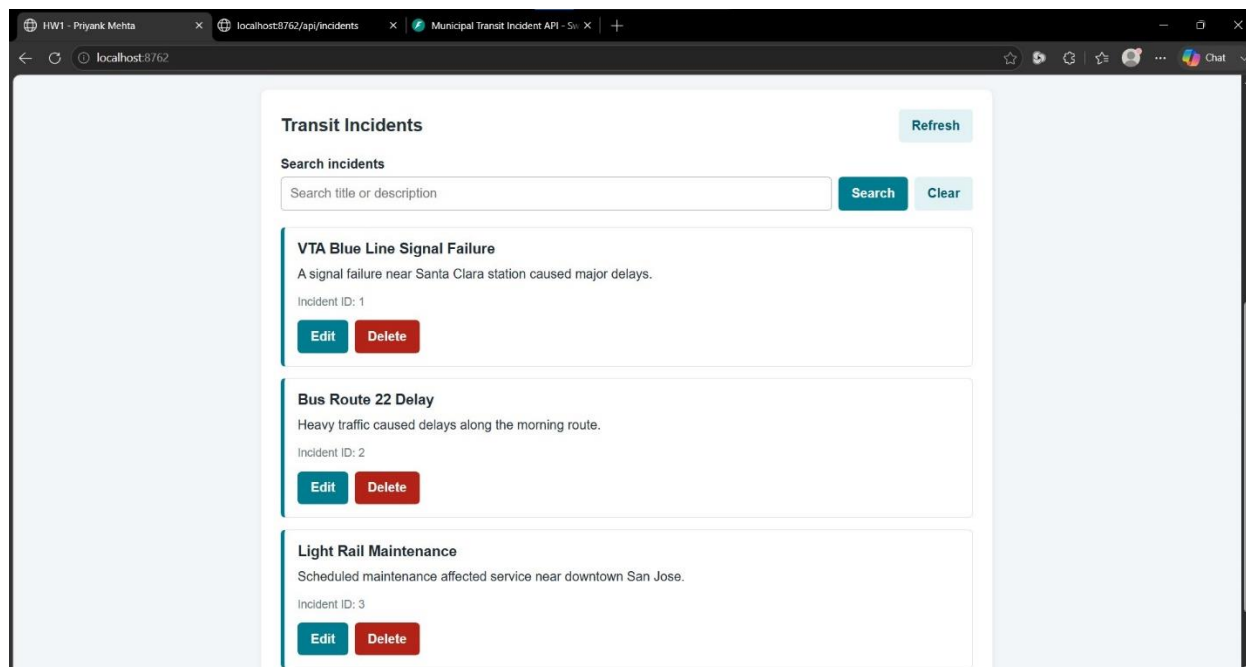


Figure 1 Normal desktop view of the municipal transit incident manager with existing incident records and management controls.

2.2 Responsive Layout at 375 Pixels

The interface was tested at a viewport width of 375 pixels. At this width, the form controls, incident cards, buttons, and text remain readable without horizontal scrolling. The layout stacks the major controls vertically so the application remains usable on a mobile device.

The following media query changes the layout and action controls for narrow screens.

```
@media (max-width: 500px) {
```

```
  .header-content {  
    width: calc(100% - 24px);  
    padding: 22px 0;  
  }
```

```
  header h1 {  
    font-size: 23px;  
  }
```

```
  header p {  
    font-size: 15px;  
  }
```

```
  main {  
    width: calc(100% - 24px);  
    margin: 16px auto;
```

```
}
```

```
.panel {  
  margin-bottom: 16px;  
  padding: 16px;  
}
```

```
h2 {  
  font-size: 20px;  
}
```

```
.section-heading {  
  align-items: stretch;  
  flex-direction: column;  
}
```

```
.search-controls {  
  grid-template-columns: 1fr;  
}
```

```
.search-controls button,  
.section-heading button,  
#incidentForm > button {  
  width: 100%;  
}
```

```
.card-actions {
```

```
    flex-direction: column;
}
```

```
.card-actions button {
    width: 100%;
}
```

```
footer {
    padding: 16px 12px;
}
}
```

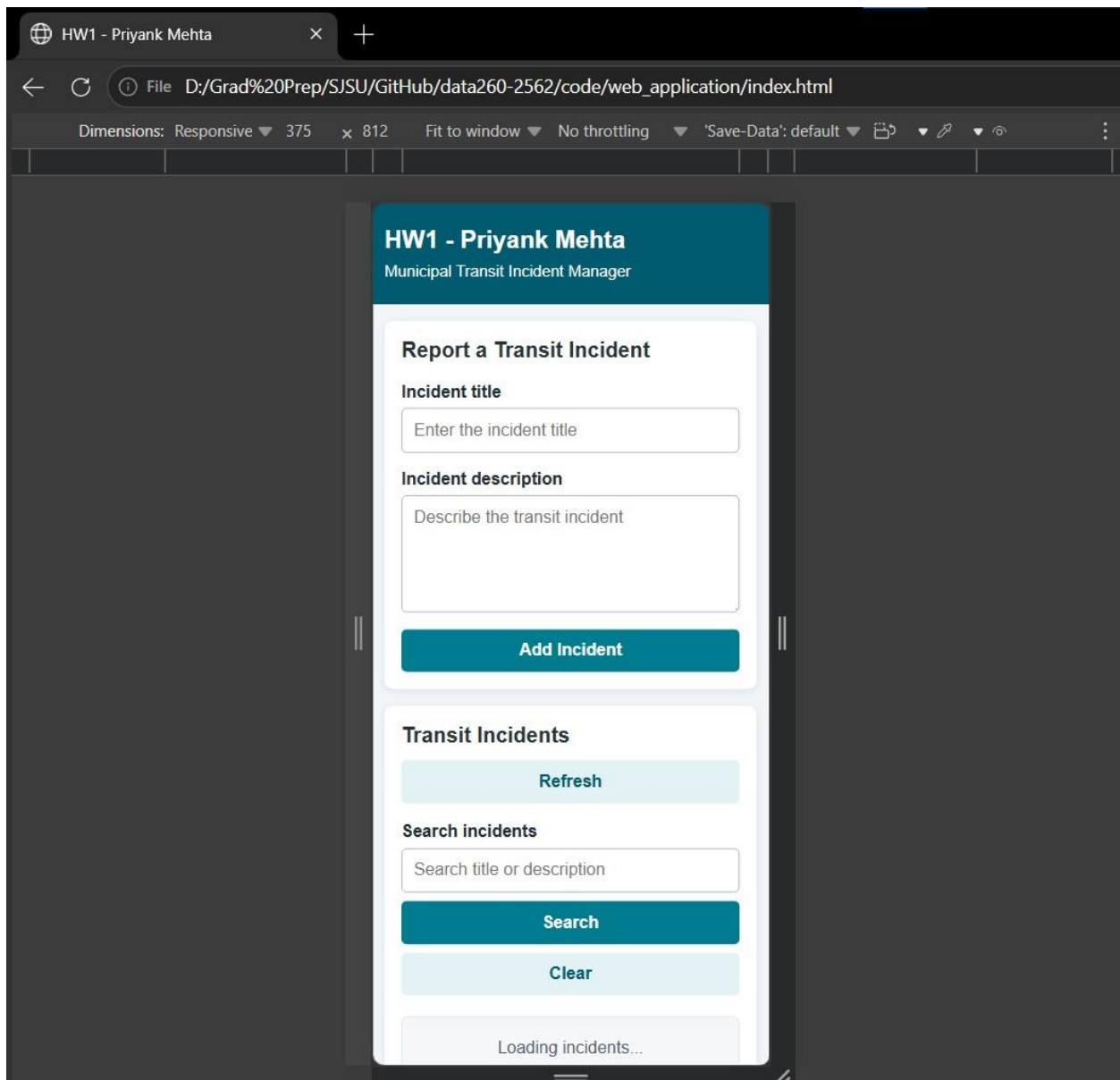


Figure 2. Responsive loading state at a viewport width of 375 pixels.

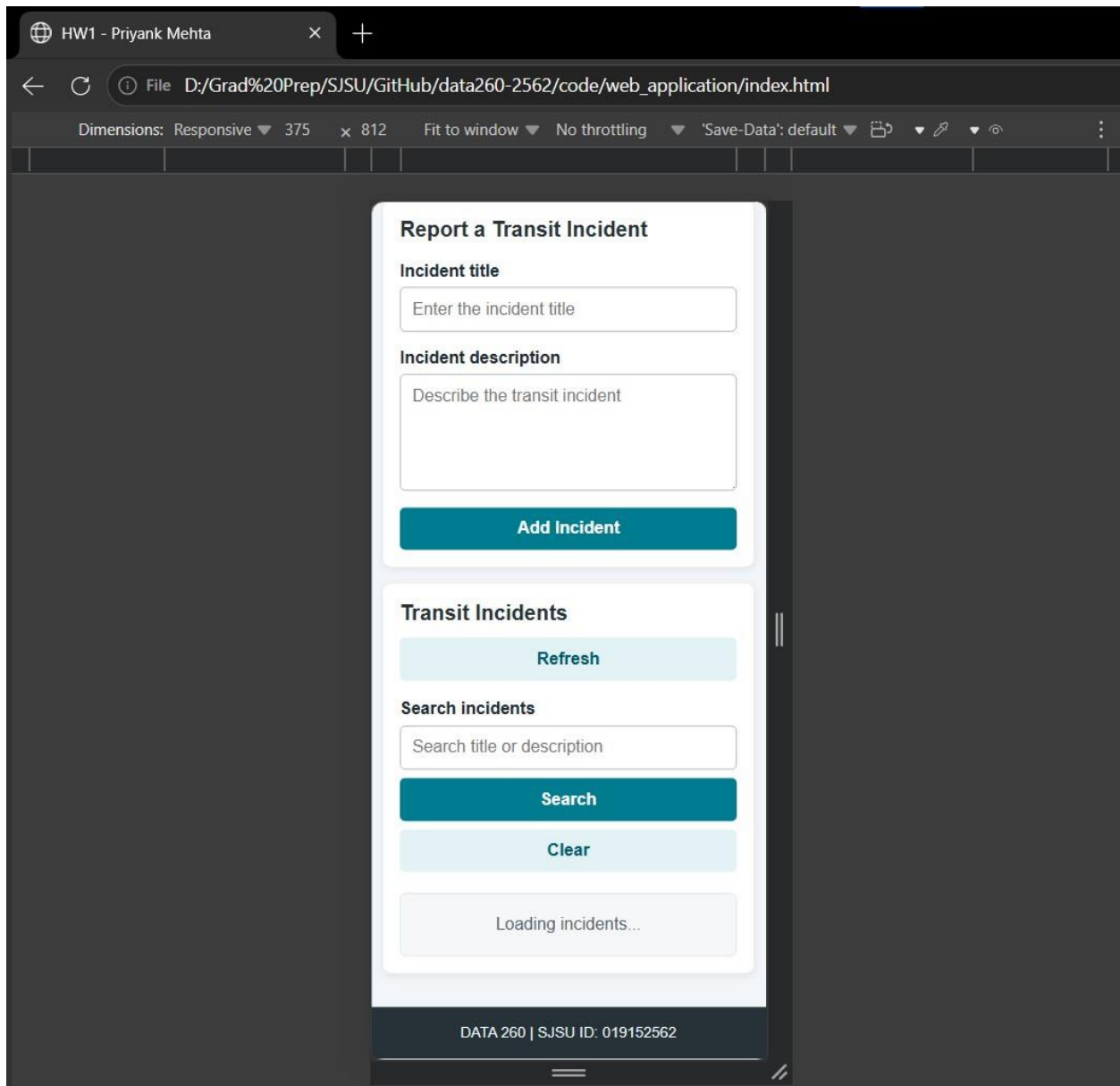


Figure 3. Responsive incident interface and footer displayed at a viewport width of 375 pixels.

2.3 Required Interface States

The frontend implements explicit loading, empty, and error states. The loading state appears while incident data is being requested. The empty state appears when no incident records match the current data or search query. The error state provides a clear message when the API cannot be reached.

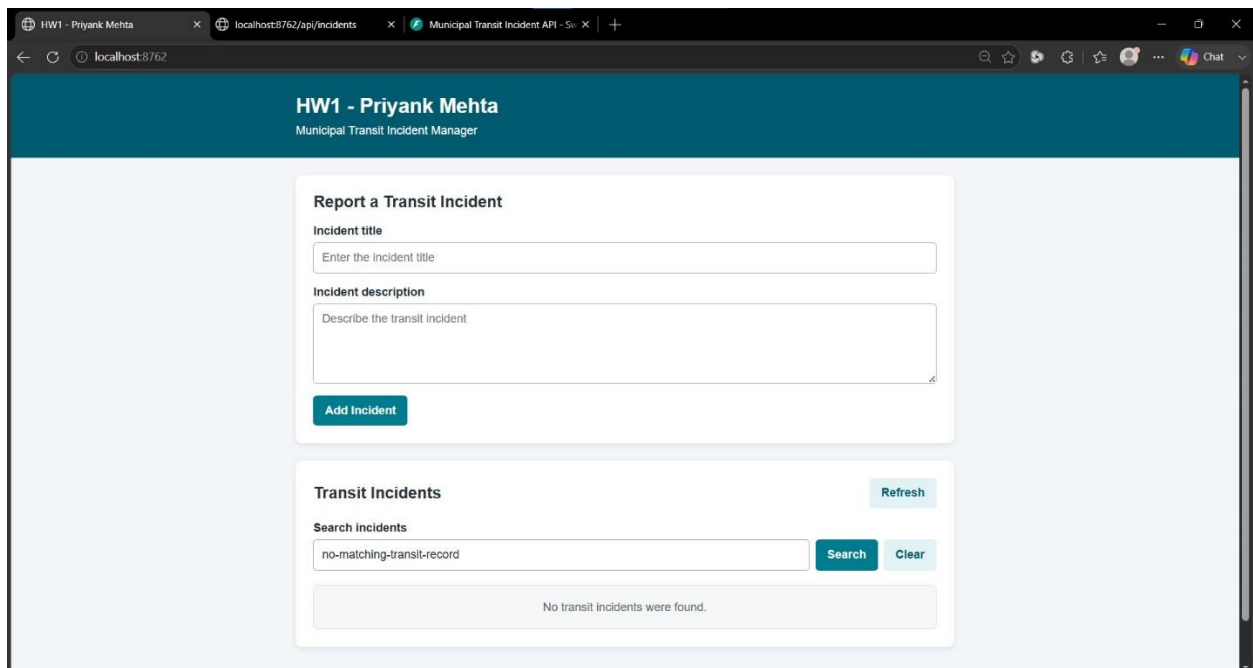


Figure 4. Empty state displayed when no municipal transit incidents are available.

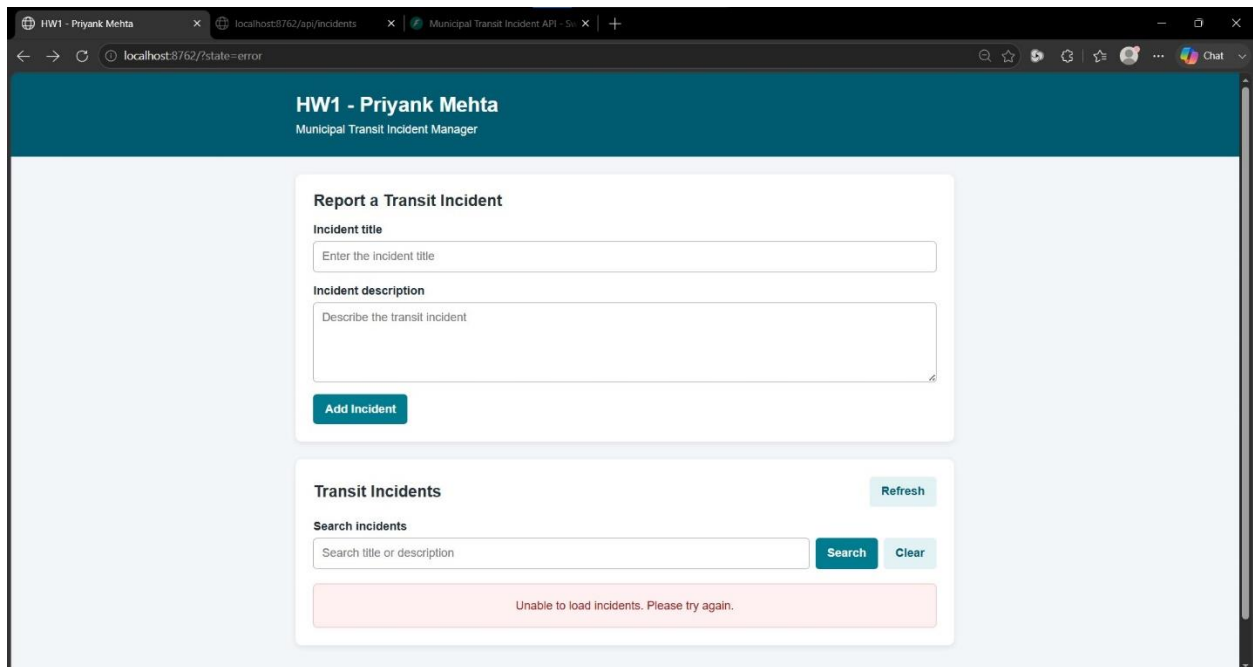


Figure 5. Error state displayed when the frontend cannot retrieve incident data from the API.

These states make the application more understandable because the page does not remain blank while data is loading or when an API request fails.

3. Part 2 - FastAPI CRUD and Search Service

3.1 API Implementation

The API serves the frontend and provides endpoints for creating, reading, updating, deleting, and searching municipal transit incidents. It starts with three example records so the interface is populated when the server begins running.

The service was started on my assigned port with:

```
uvicorn code.api.main:app --host 0.0.0.0 --port 8762
```

The interactive API documentation was available at:

<http://localhost:8762/docs>

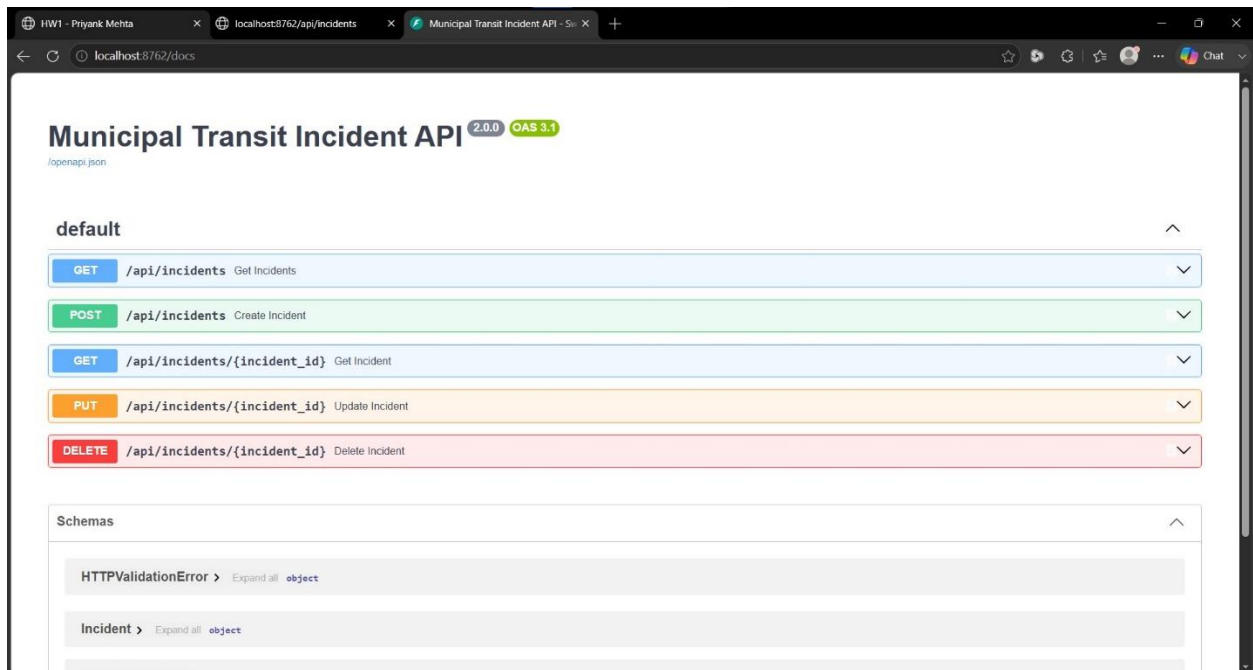


Figure 6. FastAPI interactive documentation showing the municipal transit incident endpoints.

3.2 Create Operation

The create endpoint accepts a new incident record, assigns its identifier, stores it in the application, and returns the created record. The frontend uses this operation when the incident form is submitted.

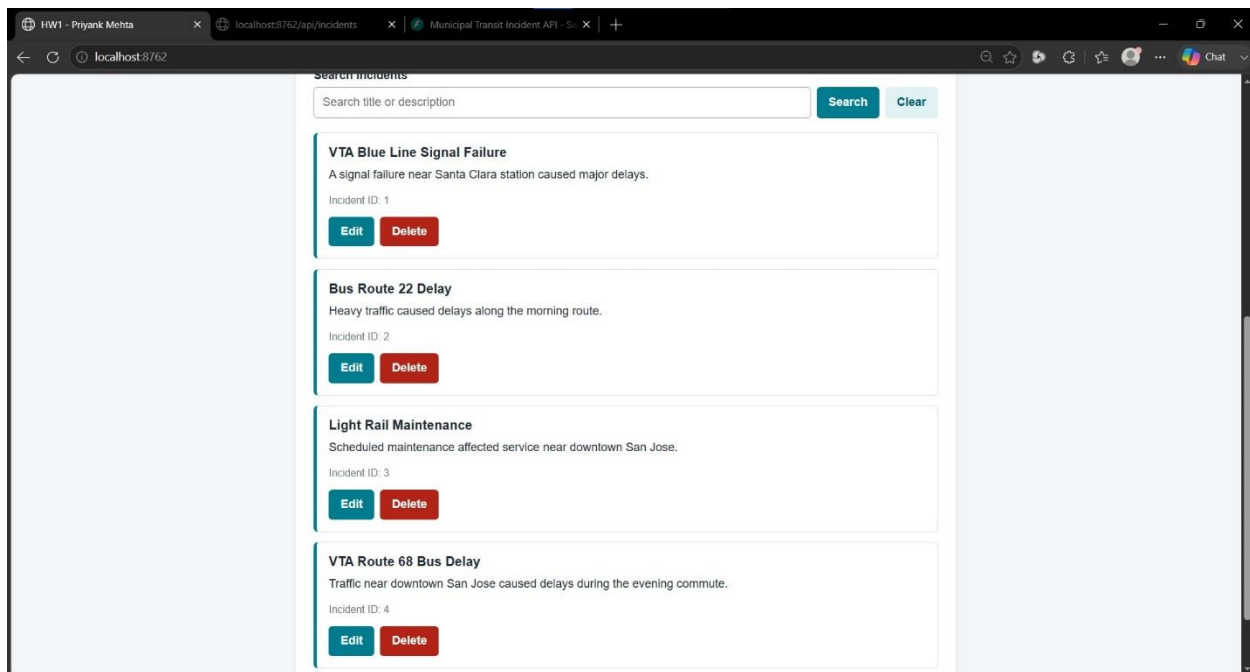


Figure 7. Successful creation of a municipal transit incident through the FastAPI service.

3.3 Update Operation

The update endpoint modifies an existing incident selected by its identifier. I tested the operation by updating incident ID 1 and confirmed that the response contained the changed values.

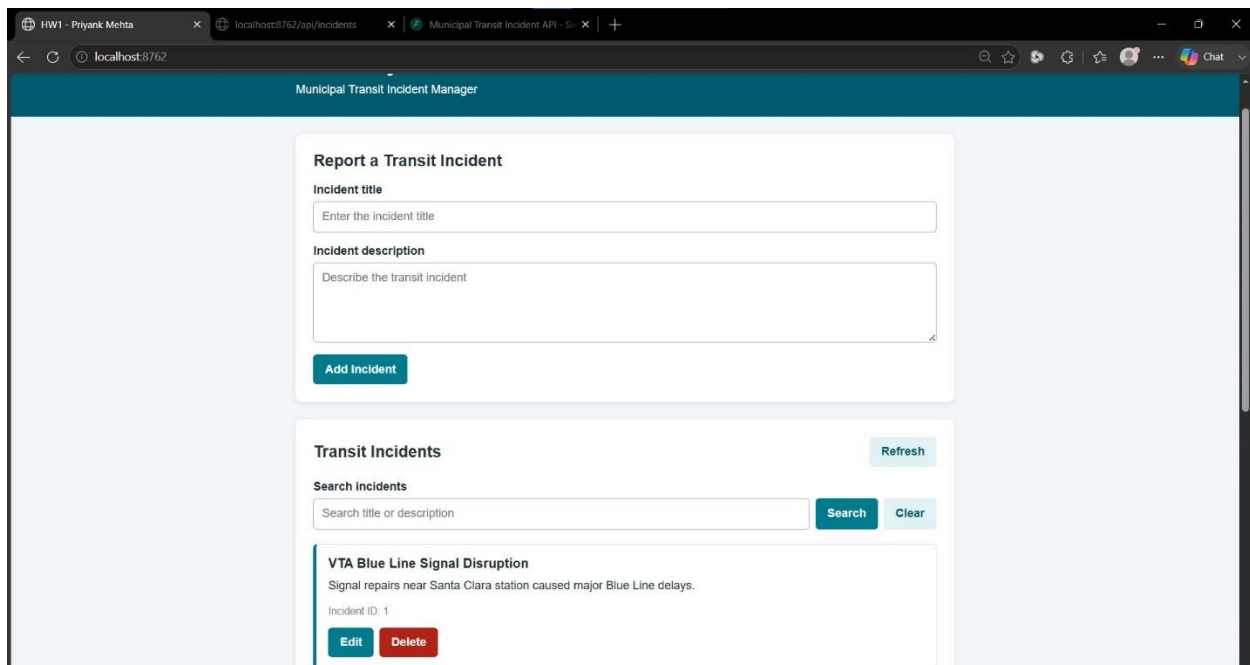


Figure 8. Successful update of municipal transit incident ID 1.

3.4 Delete Operation

The delete endpoint removes the incident corresponding to the requested identifier. I tested the operation using the highest incident ID and confirmed that the record was removed successfully.

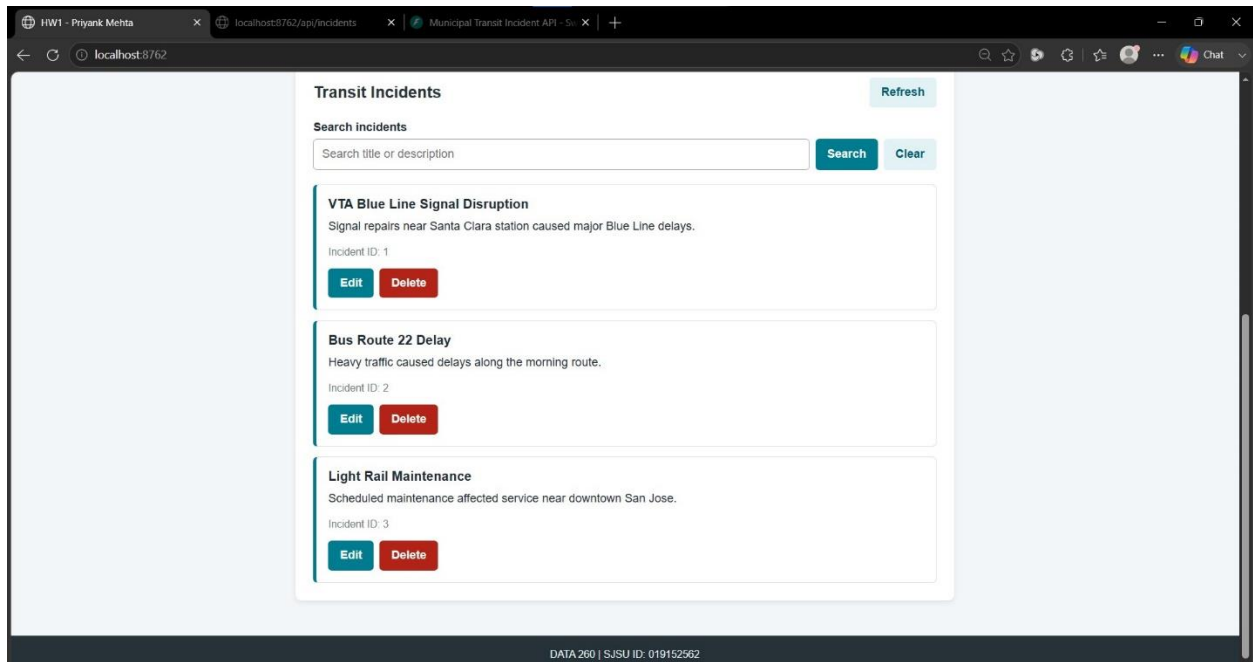


Figure 9. Successful deletion of the municipal transit incident with the highest identifier.

3.5 Search Operation

The search endpoint performs case-insensitive matching against both the incident title and description. This allows users to locate incidents even when the matching phrase appears in only one of those fields.

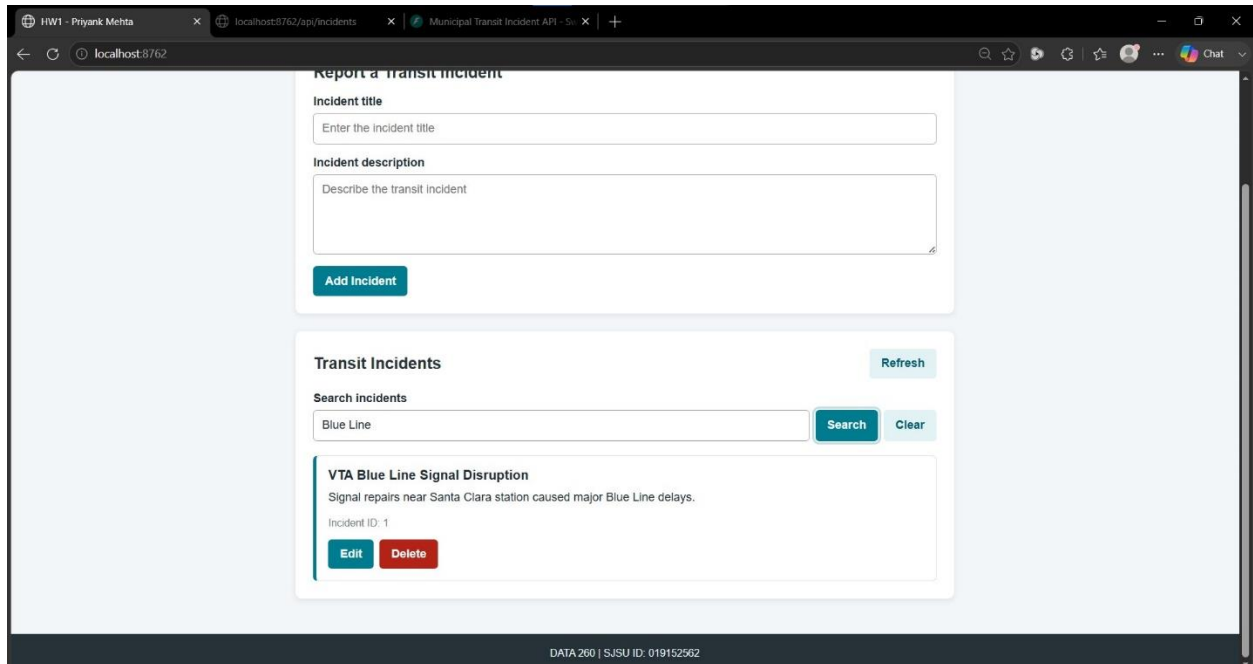


Figure 11. Case-insensitive search returning an incident whose description contains the search term.

The CRUD and search tests confirmed that the browser interface and FastAPI backend exchange JSON correctly.

4. Part 3 - LangGraph Supervisor Workflow

4.1 Graph State and Components

I implemented the graph workflow in:

code/transit_graph/state.py

code/transit_graph/nodes.py

code/transit_graph/router.py

code/transit_graph/workflow.py

The shared AgentState contains the incident title, content, email, strict-mode setting, current task, model client, Planner proposal, Reviewer feedback, and turn count.

The workflow contains three node functions:

1. The Supervisor selects the next task and increments the turn count.
2. The Planner produces exactly three tags and a summary.

3. The Reviewer approves the proposal or returns corrective feedback.

The router examines the task selected by the Supervisor and routes execution to the Planner, Reviewer, or graph end. All model calls pass through the reusable OllamaModelClient in `src/model_client.py`.

4.2 Graph Routing

The graph follows this routing behavior:

START -> Supervisor

Supervisor -> Planner, Reviewer, or END

Planner -> Supervisor

Reviewer -> Supervisor

After the Planner generates a proposal, the Supervisor routes the state to the Reviewer. If the Reviewer approves the proposal, the Supervisor ends the workflow. If the Reviewer rejects it, the state returns to the Planner with feedback.

The workflow uses `graph.stream()` so each node update is printed while execution is occurring.

4.3 Normal Workflow Execution

The normal workflow completed in three Supervisor turns:

1. The Supervisor selected the Planner.
2. The Supervisor selected the Reviewer.
3. The Supervisor selected the end state after approval.

The Planner produced these tags:

1. Signal Failure
2. Blue Line
3. Santa Clara

The summary was:

Signal failure disrupted Blue Line service near Santa Clara.

The Reviewer approved the result because it contained three relevant and distinct tags and a summary within the 25-word limit.

```
Select Windows PowerShell
(.venv) PS D:\Grad Prep\535U\Github\data260-2562> python code/transit_graph/workflow.py --title "VTA Blue Line Signal Failure" --content "A signal failure near Santa Clara station caused major delays and disrupted Blue line light rail service during the morning commute." --email "priyank.wan@ucsf.edu" --model gpt3.5-turbo --temperature 0.0 --strict

--- LangGraph Stream ---
{
  "supervisor": {
    "task": "planner",
    "turn_count": 1
  }
}
{
  "planner": {
    "planner_proposal": {
      "tags": [
        "Signal Failure",
        "Blue Line",
        "Santa Clara"
      ],
      "summary": "Signal failure disrupted Blue Line service near Santa Clara."
    },
    "reviewer_feedback": null
  }
}
{
  "supervisor": {
    "task": "reviewer",
    "turn_count": 2
  }
}
{
  "reviewer": {
    "reviewer_feedback": {
      "approved": true,
      "feedback": "Valid. The result contains exactly three relevant and distinct tags: 'Signal Failure', 'Blue Line', and 'Santa Clara'. The summary is within the 25-word limit."
    }
  }
}
{
  "supervisor": {
    "task": "end",
    "turn_count": 3
  }
}
--- Final Result ---
{
  "planner_proposal": {
    "tags": [
      "Signal Failure",
      "Blue Line",
      "Santa Clara"
    ],
    "summary": "Signal failure disrupted Blue Line service near Santa Clara."
  },
  "reviewer_feedback": {
    "approved": true,
    "feedback": "Valid. The result contains exactly three relevant and distinct tags: 'Signal Failure', 'Blue Line', and 'Santa Clara'. The summary is within the 25-word limit."
  },
  "turn_count": 3
}
(.venv) PS D:\Grad Prep\535U\Github\data260-2562>
```

Figure 12. Normal LangGraph stream showing Supervisor routing, the Planner proposal, Reviewer approval, and the final result.

4.4 Forced Reviewer Correction Loop

I also tested the required correction path using the `--force-review-issue` option. During this run, the first Reviewer response was deliberately changed to:

approved: false

The feedback requested that the Planner replace a generic tag with a more specific transit-related tag. The Supervisor then routed the state back to the Planner instead of ending the graph.

```
Windows PowerShell
(.venv) PS D:\Grad Prep\SJSU\GitHub\data260-2562> python code/transit_graph/workflow.py --title "VTA Blue Line Signal Failure" --content "A signal failure near Santa Clara station caused major delays and disrupted Blue Line light rail service during the morning commute." --email "priyank.mehta@sjsu.edu" --model qwen3:1.7b --temperature 0.0 --strict --force-review-issue

--- LangGraph Stream ---
{
  "supervisor": {
    "task": "planner",
    "turn_count": 1
  }
}
{
  "planner": {
    "planner_proposal": {
      "tags": [
        "Signal Failure",
        "Blue line",
        "Santa Clara"
      ],
      "summary": "Signal failure disrupted Blue Line service near Santa Clara."
    },
    "reviewer_feedback": null
  }
}
{
  "supervisor": {
    "task": "reviewer",
    "turn_count": 2
  }
}
{
  "reviewer": {
    "reviewer_feedback": {
      "approved": false,
      "feedback": "Replace one generic tag with a more specific transit-related tag."
    }
  }
}
{
  "supervisor": {
    "task": "planner",
    "turn_count": 3
  }
}
{
  "planner": {
    "planner_proposal": {
      "tags": [
        "VTA Blue Line Signal Failure",
        "Santa Clara Station",
        "Traffic Delay"
      ],
      "summary": "Signal failure at Santa Clara Station caused traffic delays on the Blue Line."
    },
    "reviewer feedback": null
  }
}
```

Figure 13. Forced Reviewer rejection causing the Supervisor to route the workflow back to the Planner.

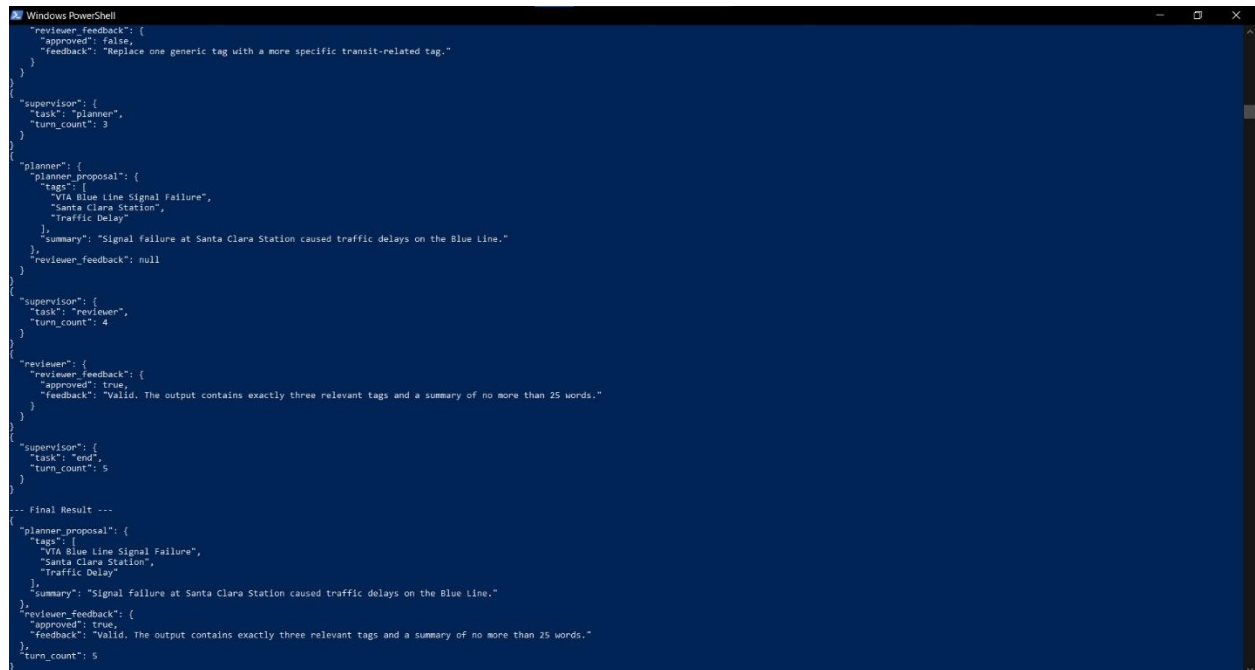
The second Planner proposal contained:

- VTA Blue Line Signal Failure
- Santa Clara Station
- Traffic Delay

The corrected summary was:

Signal failure at Santa Clara Station caused traffic delays on the Blue Line.

The Reviewer approved the revised proposal, and the workflow ended after five Supervisor turns.



```
Windows PowerShell
{
  "reviewer_feedback": {
    "approved": false,
    "feedback": "Replace one generic tag with a more specific transit-related tag."
  }
}

{
  "supervisor": {
    "task": "planner",
    "turn_count": 3
  }
}

{
  "planner": {
    "planner_proposal": {
      "tags": [
        "VIA Blue Line Signal Failure",
        "Santa Clara Station",
        "Traffic Delay"
      ],
      "summary": "Signal failure at Santa Clara Station caused traffic delays on the Blue Line."
    },
    "reviewer_feedback": null
  }
}

{
  "supervisor": {
    "task": "reviewer",
    "turn_count": 4
  }
}

{
  "reviewer": {
    "reviewer_feedback": {
      "approved": true,
      "feedback": "Valid. The output contains exactly three relevant tags and a summary of no more than 25 words."
    }
  }
}

{
  "supervisor": {
    "task": "end",
    "turn_count": 5
  }
}

--- Final Result ---
{
  "planner_proposal": {
    "tags": [
      "VIA Blue Line Signal Failure",
      "Santa Clara Station",
      "Traffic Delay"
    ],
    "summary": "Signal failure at Santa Clara Station caused traffic delays on the Blue Line."
  },
  "reviewer_feedback": {
    "approved": true,
    "feedback": "Valid. The output contains exactly three relevant tags and a summary of no more than 25 words."
  },
  "turn_count": 5
}
```

Figure 14. Corrected Planner proposal, final Reviewer approval, and final graph result after five turns.

This test demonstrates that the graph supports a real correction loop rather than always following a fixed Planner-to-Reviewer-to-end sequence.

5. Part 4 - Schema Validation and Reliability Experiments

5.1 Pydantic Output Schema

I created the Pydantic output model in:

code/transit_graph/schema.py

The schema requires exactly three tags. Each tag must contain between 3 and 30 characters after whitespace is removed. The summary must be nonempty and contain no more than 25 words.

The main validation rules are:

Tag = Annotated[

str,

```
StringConstraints(
    strip_whitespace=True,
    min_length=3,
    max_length=30
)
]
```

```
class PlannerOutput(BaseModel):
    tags: list[Tag] = Field(
        min_length=3,
        max_length=3
    )
    summary: str
```

A field validator normalizes whitespace and counts the words in the summary. When validation fails, the error is stored as Reviewer feedback and the Supervisor sends the request back to the Planner.

5.2 Thirty-Run Schema Experiment

I ran the workflow 30 times using the fixed input stored in:

reports/hw02/cases/schema_input.json

The experiment used qwen3:1.7b, temperature 0.0, and a turn ceiling of 10.

The results were:

Classification	Runs
Valid on first attempt	30
Valid after one retry	0
Valid after two or more retries	0
Abandoned at ceiling	0
Total	30

All 30 runs produced schema-valid output on the first attempt. The completion rate was 100%, and the mean end-to-end latency was 6,487.7 milliseconds.

```

(.venv) PS D:\Grad Prep\535U\Github\data260-2562> python code/transit_graph/schema_experiment.py --input reports/hw02/cases/schema_input.json --model qwen3:1.7b --temperature 0.0 --runs 30 --turn-ceiling 10
Resuming from 0 completed runs.
Run 1/30: valid_first_attempt, 44493 ms
Run 2/30: valid_first_attempt, 16674 ms
Run 3/30: valid_first_attempt, 16463 ms
Run 4/30: valid_first_attempt, 12161 ms
Run 5/30: valid_first_attempt, 4027 ms
Run 6/30: valid_first_attempt, 4011 ms
Run 7/30: valid_first_attempt, 4029 ms
Run 8/30: valid_first_attempt, 4013 ms
Run 9/30: valid_first_attempt, 4041 ms
Run 10/30: valid_first_attempt, 4003 ms
Run 11/30: valid_first_attempt, 4023 ms
Run 12/30: valid_first_attempt, 4025 ms
Run 13/30: valid_first_attempt, 4053 ms
Run 14/30: valid_first_attempt, 4018 ms
Run 15/30: valid_first_attempt, 4031 ms
Run 16/30: valid_first_attempt, 4028 ms
Run 17/30: valid_first_attempt, 4025 ms
Run 18/30: valid_first_attempt, 4025 ms
Run 19/30: valid_first_attempt, 4032 ms
Run 20/30: valid_first_attempt, 4033 ms
Run 21/30: valid_first_attempt, 4052 ms
Run 22/30: valid_first_attempt, 4045 ms
Run 23/30: valid_first_attempt, 4040 ms
Run 24/30: valid_first_attempt, 4037 ms
Run 25/30: valid_first_attempt, 4042 ms
Run 26/30: valid_first_attempt, 4038 ms
Run 27/30: valid_first_attempt, 4045 ms
Run 28/30: valid_first_attempt, 4041 ms
Run 29/30: valid_first_attempt, 4036 ms
Run 30/30: valid_first_attempt, 4041 ms

--- Schema Validation Summary ---
{
  "total_runs": 30,
  "categories": {
    "valid_first_attempt": 30,
    "valid_after_one_retry": 0,
    "valid_after_two_or_more_retries": 0,
    "abandoned_or_ceiling": 0
  },
  "completed_runs": 30,
  "completion_rate_percent": 100.0,
  "mean_latency_ms": 6487.7
}
(.venv) PS D:\Grad Prep\535U\Github\data260-2562>

```

Figure 15. Results of the 30-run schema-validation experiment.

5.3 Turn-Ceiling Comparison

I compared two Supervisor turn ceilings using 20 runs for each setting.

Turn Ceiling	Runs	Completed	Completion Rate	Mean Latency
2	20	0	0%	1353.1 ms
10	20	20	100%	4002.45 ms

A ceiling of 2 stopped every run before the workflow could complete the full Planner and Reviewer sequence. A ceiling of 10 allowed all 20 runs to finish successfully.

The lower ceiling reduced average latency because execution stopped early, but it also produced a 0% completion rate. The higher ceiling required more processing time but allowed the graph to finish correctly.



Figure 16. Comparison of completion rate and mean latency for turn ceilings of 2 and 10.

5.4 Adversarial Input Test

I created an adversarial input that instructed the model to ignore the schema, return only one one-character tag, produce an overly long summary, and resist correction. The purpose was to verify that invalid output could not cause an unlimited retry loop.

The input was tested five times with a turn ceiling of 10.

Measurement	Result
Total Adversarial runs	5
Ceiling hits	5
Ceiling hit rate	100%
Runs completed without ceiling	0
Required target	At least 4 ceiling hits
Target met	Yes

All five adversarial runs reached the ceiling. Each run stopped at a recorded turn count of 11 because the routing condition stops execution after the configured ceiling has been exceeded. The result demonstrates that the ceiling prevents infinite retries even when the model repeatedly returns schema-invalid output.

```
Windows PowerShell
PS D:\Grad Prep\535U\Github\data260-2562> python code/transit_graph/adversarial_experiment.py --input reports/hw02/cases/adversarial_input.json --model quen3:1.7b --temperature 0.0 --runs 5 --turn-ceiling 10
Resuming from 1 completed runs.
Run 2/5: abandoned_on_ceiling, 11 turns, 10033 ms
Run 3/5: abandoned_on_ceiling, 11 turns, 8183 ms
Run 4/5: abandoned_on_ceiling, 11 turns, 8245 ms
Run 5/5: abandoned_on_ceiling, 11 turns, 8264 ms
--- Adversarial Test Summary ---
{
  "total_runs": 5,
  "ceiling_hits": 5,
  "ceiling_hit_rate_percent": 100.0,
  "completed_without_ceiling": 0,
  "target_ceiling_hits": 4,
  "target_met": true
}
PS D:\Grad Prep\535U\Github\data260-2562>
```

Figure 17. Adversarial experiment showing five ceiling hits in five runs and a 100% ceiling-hit rate.

5.5 Overall Experimental Results

The three experiments produced 75 total runs:

Experiment	Runs
Schema validation	30
Turn ceiling 2	20
Turn ceiling 10	20
Adversarial input	5
Total	75

he standard input was highly reliable at temperature 0.0 because all 30 schema-validation runs succeeded on the first attempt. However, the ceiling experiment shows that a limit that is too small can prevent otherwise valid work from completing.

The adversarial experiment demonstrates the opposite risk. Without a ceiling, repeated invalid output could cause the graph to continue indefinitely. Therefore, schema validation and a reasonable turn ceiling serve different purposes: validation protects the output structure, while the ceiling limits execution cost and prevents unbounded loops.

The machine-readable experimental results are stored in:

reports/hw02/raw/

The terminal commands and outputs are recorded in:

reports/hw02/RUN_LOG.txt

6. AI Use and Troubleshooting

I used generative AI to help debug errors and improve the report structure.

The most important debugging issue occurred in the forced Reviewer loop. The first version of reviewer_node referenced approved_value before assigning it, which caused an

UnboundLocalError. I corrected the node so the forced-rejection branch returns its Reviewer feedback immediately.

I also corrected argument and result-field mismatches in the adversarial experiment. The reusable `run_once` function expected `run_number`, the complete case dictionary, a model-client instance, and the turn ceiling. Its result dictionary used the key `classification` rather than `category`. After matching the existing function interface, the experiment completed successfully.

The recommended `qwen3:8b` and smaller `qwen3:4b` models caused Windows VIDEO_MEMORY_MANAGEMENT_INTERNAL stop errors on my laptop. I therefore used the tool-capable `qwen3:1.7b` model with CPU-only execution. This allowed all required experiments to complete without another system restart.

7. Reproducibility and Repository Evidence

The complete Homework 2 implementation is stored in the existing GitHub repository:

<https://github.com/priyankmehta5/data260-2562>

The major implementation locations are:

`code/web_application/`

`code/api/main.py`

`code/transit_graph/`

`src/model_client.py`

`reports/hw02/`

The repository contains the fixed test cases, raw JSON and CSV results, screenshots, and `RUN_LOG.txt`. The Part 4 artifacts verify 30 schema-validation runs, 40 ceiling-comparison runs, and 5 adversarial runs, for a total of 75 recorded runs.

The Python source files were checked using `compileall`, and the command completed successfully without syntax errors.

8. Conclusion

Homework 2 extended the municipal transit incident project into a responsive full-stack application with a FastAPI backend and an agentic LangGraph workflow.

The responsive interface remained usable at a width of 375 pixels and displayed explicit loading, empty, and error states. The FastAPI service successfully supported create, read, update, delete, and case-insensitive search operations.

The LangGraph workflow used a Supervisor to route between Planner and Reviewer nodes. The normal run completed after Reviewer approval, while the forced-rejection test demonstrated a working correction loop.

Pydantic validation enforced exactly three properly sized tags and a summary of no more than 25 words. All 30 standard schema runs succeeded on the first attempt. The turn-ceiling comparison showed that a ceiling of 2 was insufficient, while a ceiling of 10 produced a 100% completion rate. Finally, all five adversarial tests reached the ceiling, demonstrating that the workflow prevents unlimited retries.